Session 4: From Data to Deliverable

Business Intelligence & AI

Kerry Back, Rice University

Homework Debrief

Best and Worst Answers

Share Your Best Answer

- What question did you ask?
- What made the answer good?
- Would you trust it without further checking?

Share Your Worst Answer

- What went wrong?
- Was the answer wrong, misleading, or just incomplete?
- What verification would have caught it?

Applying Maker/Checker

Low Stakes

- Internal exploration, brainstorming
- “How many customers does each division have?”
- Quick sanity check: do the numbers look reasonable?

Medium Stakes

- Team presentations, internal reports
- “Revenue per employee by division”
- Verify: check the SQL, confirm date ranges, spot-check totals

High Stakes

- Board decks, regulatory filings
- “Quarterly executive summary for the CEO”
- **Independent verification of every claim**

Full Pipeline Demo

From Questions to Deliverables

In Sessions 1–3, we asked questions and got answers. Today we chain those steps into **complete reporting pipelines** — from raw data to polished, board-ready documents.

> One prompt. Six systems queried. Charts generated. Narrative written. Document assembled. **That is the pipeline.**

The Pipeline Steps

1

Query

Pull from 6 source systems

→

2

Merge

Join and deduplicate

→

3

Compute

Calculate KPIs and variances

→

4

Chart

Generate visualizations

→

5

Narrate

Write analysis text

→

6

Assemble

Format final document

Live Demo: “Prepare a Quarterly Executive Summary”

The prompt

```
Prepare a quarterly executive summary of XYZ Corp performance ---
revenue by division, headcount efficiency, top customer risks, and
supply chain issues. Include charts and a narrative summary.
```

Demo Result: Q4 Executive Summary

Division	Q4 2025	Q3 2025	QoQ Growth	Active Customers
Industrial	$5,891,237	$5,482,103	+7.5%	91
Energy	$3,412,891	$3,187,445	+7.1%	65
Safety	$1,147,892	$1,023,556	+12.1%	37
Total	**$10,452,020**	**$9,693,104**	**+7.8%**	**193**

Safety posted the strongest growth rate at **12.1%**. But Energy has a **34% customer concentration risk** in CAT Energy Services — a single-point-of-failure.

Now Iterate

Draft 1: Data Summary

- Revenue by division, headcount, growth rates
- Comprehensive but flat
- “Summarize Q4 performance”

Draft 2: Traffic-Light Format

- Red / Yellow / Green status by division
- Flags anything >10% off plan
- “Traffic-light format, flag >10% off plan”

Draft 3: Executive Brief

- Leads with the biggest risk
- Ends with 3 recommended actions
- **“One page, lead with risk, end with actions”**

Same data. Same agent. **The prompt was the variable that mattered.**

Three Iterations Compared

Draft	Prompt Addition	Output	Audience
1	“Summarize Q4 performance”	Data dump — no prioritization	Analyst
2	+ “Traffic-light, flag >10% off plan”	Structured dashboard — scannable	Manager
3	+ “One page, lead with risk, end with actions”	Executive brief — board-ready	C-suite

The prompt is not just a question — it is a **specification**. More specific prompts produce more useful deliverables. This is the skill you developed in Session 2, now applied to enterprise data.

How the Agent Loop Works

The Core Loop

1

System Prompt

Schema description: what tables, columns, and relationships exist

→

2

Tool Call

LLM writes SQL and requests execution

→

3

Execute & Return

System runs SQL, sends results back to LLM

→

4

Test & Review

LLM checks results — correct? try again? deliver?

This is a **test-and-review cycle**, not a one-shot generation. The LLM writes code, the system executes it, the LLM reviews the results, and decides: deliver or try again.

The System Prompt Is Everything

What the LLM sees before your question

```
You are a data analyst for XYZ Corp, a $500M B2B distributor.

Available databases:
- sf_opportunities: Salesforce CRM (Industrial division)
  Columns: OpportunityId, AccountName, Amount, CloseDate, Stage...
- legacy_orders: Legacy CRM (Energy division)
  Columns: order_id, customer_name, order_amount, order_date (VARCHAR!)...
- hs_deals: HubSpot CRM (Safety division)
  Columns: deal_id, company_name, amount, close_date...
- wd_employees: Workday HR
  Columns: employee_id, division, department, hire_date, salary...

IMPORTANT: legacy_orders stores dates as MM/DD/YYYY text strings.
```

The Test-and-Review Cycle

What Makes This Different from a Dashboard

- Dashboards run **pre-written** queries
- If the query is wrong, you get wrong answers forever
- Changing the question means changing the code
- No self-correction capability

What the Agent Does

- Writes a **new query** for every question
- Reviews results: empty set? error? retry
- Can rewrite SQL 2-3 times before delivering
- **Self-corrects — but not infallibly**

From Ad Hoc to Automated

The Production Pattern

What You Have Been Doing

- Type a question in a chat interface
- Get an answer in real time
- Useful — but manual and ad hoc
- You still have to remember to ask

What Production Looks Like

- Same agent loop, triggered by a **schedule**
- Monday morning dashboard refresh
- Monthly board deck, weekly sales forecast
- **Error handling + version control + human review gates**

Real-World Benchmark: HPE

“Alfred” — HPE’s AI Agent

HPE’s CFO Marie Myers deployed an AI agent that replaced a **100-slide weekly performance review** that consumed hundreds of person-hours. The agent queries financial systems, generates analysis, and produces executive summaries — the same pipeline you just saw.

40%

faster reporting cycle

25%

lower processing costs

90%

reduction in manual prep

Live Walkthrough: Building an Agent

From English to Working Code

What You Are About to See

- The instructor opens a terminal on the AI+Code Lab
- Types English instructions to Claude Code: “Build a Python script that takes a question, sends it to Claude with a schema description, gets SQL back, runs it, and prints the answer”
- Claude Code generates ~50 lines of Python
- We run it. We ask a question. We get an answer.

> The entire agent — from blank file to working code — takes about **5 minutes**. This is the “your developer could do this” moment.

The 50 Lines vs. the 3,000 Lines

50 Lines: The Prototype

- System prompt with schema
- Tool call mechanism (SQL execution)
- Basic response formatting
- Works for one database, one user
- **This is what you just saw**

3,000 Lines: Production

- Multiple database connections
- Authentication and access control
- Error handling and retry logic
- Logging, monitoring, audit trail
- Web interface, API, scheduling
- **This is the XYZ Corp Data Simulation**

The core is simple. Production is engineering. Understanding the difference helps you evaluate vendors and scope internal builds.

From Pipeline to Dashboard

The reporting pipeline produces a **document** — a PDF, a memo, a deck. But what if you need something **interactive** — a dashboard your team can use every day, with filters, drill-downs, and live data?

> One prompt to Claude Code: “Build an interactive dashboard that shows revenue by division with filters for date range and product category.” Working app in under **5 minutes**.

Demo Walkthrough: Building a Dashboard

The prompt (typed into Claude Code on the AI+Code Lab)

```
Build an interactive dashboard that visualizes XYZ Corp data.
Include: revenue by division (bar chart), a trend line for
monthly revenue, dropdown filters for date range and product
category, and a downloadable CSV of the filtered data.
```

What Claude Code Produces

- A single Python file (~80 lines) using a web framework, Pandas, and Plotly
- Data loading from the XYZ Corp dataset
- Interactive dropdown filters for division, date range, and product category
- Bar chart of revenue by division, trend line of monthly revenue
- A “Download CSV” button for the filtered dataset
- **Result:** a working web app you can open in a browser immediately

The SaaSpocalypse in Reverse

> AI kills dashboards. AI also **builds better ones faster**.

The dashboard that used to take a team two weeks to spec, design, and deploy can now be prototyped in minutes. The question is not whether dashboards are dead — it is **who builds them and how fast**.

Hands-On: Build Your Own Report

Three Report Exercises

Exercise 1: Customer Risk Report

- Combine CRM pipeline data with support ticket trends
- Which customers are high-revenue AND high-complaint?
- Include a chart and a recommendation

Exercise 2: Division P&L Comparison

- Revenue by division from CRMs
- Headcount from Workday
- Revenue per employee with commentary
- Compare divisions head-to-head

Exercise 3: Board Memo

- Q3 vs Q4 performance across all divisions
- Charts, narrative, risk flags
- **Iterate: make it one page, lead with risk**
- Use the Session 2 techniques

Iteration Tips

Getting Better Output

- **Be specific about format:** “Traffic-light dashboard” vs. “one-page executive brief”
- **Name the audience:** “For the CFO” produces different output than “for the sales team”
- **Set thresholds:** “Flag anything more than 10% off plan”
- **Ask for critique:** “What is weak about this analysis? What would a skeptical CFO push back on?”
- **Iterate:** “Lead with risk. End with three actions. Cut to one page.”

Workshop: Your Reporting Candidates

Identify Your Pipelines

Individual Exercise (12 minutes)

For **2-3 recurring reports** at your company:

1. **What is the report?** (weekly status, monthly board deck, quarterly review, etc.)
2. **What systems feed it?** (CRM, ERP, HR, finance)
3. **Who builds it today?** How long does it take?
4. **What would automation look like?** Agent queries sources, generates draft, human reviews

Report	Systems	Owner Today	Time	Automation Potential
e.g., Weekly sales forecast	*Salesforce + NetSuite*	*Sales Ops analyst*	*4 hours/week*	*High — structured data, repeating format*
e.g., Monthly board deck	*All systems*	*FP&A team*	*3 days/month*	*Medium — needs heavy human review*

Where Does the Code Run?

Three Execution Options

Cloud LLM Sandbox

- Code runs on the vendor's servers
- Easiest to set up
- Your data leaves your network
- What we use for the XYZ Corp demo (synthetic data)

Your Own Containers

- Code runs on your servers
- Data stays on-premise
- You manage infrastructure
- Higher control, higher cost

SQL-Only Mode

- LLM only generates SQL, never sees raw data
- Your system executes the SQL locally
- Minimum data exposure
- **Most restrictive but most secure**

Our demo uses synthetic data in the cloud. Production deployments keep data on-premise. Session 5 digs into these tradeoffs in detail.

Homework

This Week

Homework for Session 5

1. **Produce one complete deliverable** using the XYZ Corp Data Simulation — a memo, comparison, or risk assessment. Save or screenshot the output.
2. **Pre-reading (~35 min):**
    - Morgan Stanley case study (provided) — a regulated firm's deployment decision
    - AIMultiple: "Cloud LLM vs Local LLMs" — three deployment options
    - "What Does Your AI Actually See?" (provided) — four data security approaches
3. **Optional reading:** a16z "How 100 Enterprise CIOs Are Building and Buying Gen AI" + DUNNIXER vendor evaluation scorecard
4. **Optional:** AI+Code Lab Exercises 1–2 + Demos 1–4

Session 5 Preview

Session 5: “What Does Deployment Look Like?”

- Where does the LLM run? Cloud vs. on-premise vs. hybrid
- What does the AI actually see? Data security approaches
- Build vs. buy vs. extend — the real decision matrix
- Morgan Stanley case study discussion

Come prepared: read the case study and form an opinion.

Session 4 Summary

From Data to Deliverable

6

systems queried in one prompt

50

lines of code in the core agent loop

90%

manual prep reduction at HPE